

Java Collections Framework (Overview) ①

Ch 1 :- Need of collections Framework

To overcome the limitations of array we use the collection framework.

- * The collection in Java is a framework that provides an architecture to store and manipulate a group of objects
- * All the operations which is performed on data such as Searching, Sorting, Insertion, manipulation, deletion, etc. can be achieved by java collections.
- * Collection framework defines several classes and interfaces to represent a group of objects as a single unit.
collections are used to store, retrieve, manipulate, and communicate aggregate data

All collections frameworks contain the following:

Interfaces: These are abstract data types that represent collections. Interfaces allow collections to be manipulated independently of the details of their representation. In object-oriented lang, interfaces generally form a hierarchy

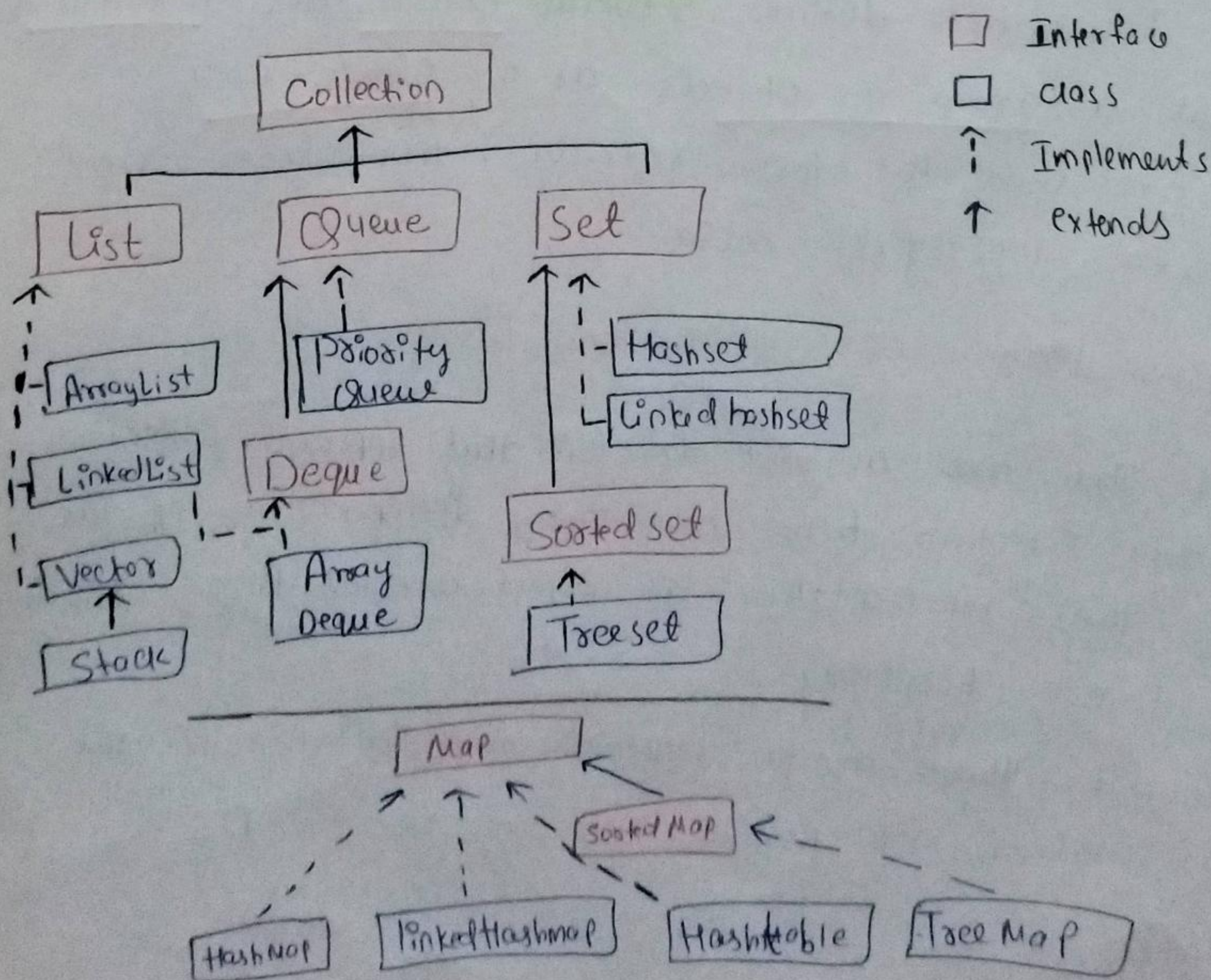
Implementations: these are the concrete implementations of the collection interfaces. In essence, they are reusable DS

Algorithms:

* Benefits of the Java Collections Framework

- 1) Reduces programming effort.
- 2) Increases performance
- 3) provides interoperability
- 4) Reduces the effort required to learn APIs
- 5) Reduces the effort required to design and implement APIs
- 6) fosters Software reuse

* Java Collections hierarchy



* List Interface which basically means those data structures in which you want to keep the items in a contiguous fashion

- ① ArrayList
Class
- dynamic size.
 - In java util package.
 - duplicate element

Syntax

`ArrayList<datatype> variable name = new ArrayList<>();`

Important method

- add()
- size()
- get()
- set()
- remove()
- add(index, element)
- addAll()
- remove(datatype.valueOf())
- clear()

Example

```
List<Integer> list = new ArrayList<>();
```

```
list.add(10);
list.add(20);
list.add(30);
sysout(list);
sysout(list.size());
list.add(40);
sysout(list);
sysout(list.get(1));
sysout(list.set(2, 143));
sysout(list.remove(1));
sysout(list.remove(1));
sysout(list);
```

O/P

```
[10 20 30]
3
[10 20 30 40]
20
30
20
[10 143 40]
```


Traversal

```
List<Integer> list = new ArrayList<>();
```

```
list.add(1);
```

```
list.add(2);
```

```
list.add(3);
```

```
list.add(4);
```

```
list.add(5);
```

```
System.out.println(list);
```

```
// for loop
```

```
for (int i = 0; i < list.size(); i++) {
```

```
    System.out.println("the element is " + list.get(i));
```

```
}
```

```
// foreach loop
```

```
for (Integer element : list) {
```

```
    System.out.println("foreach element " + element);
```

```
}
```

```
// Iterator
```

```
Iterator<Integer> it = list.iterator();
```

```
while (it.hasNext()) {
```

```
    System.out.println("hasNext " + it.next());
```

```
}
```

O/P

[1, 2, 3, 4, 5]

the element is 1

the element is 2

the element is 3

the element is 4

the element is 5

foreach element 1

foreach element 2

foreach element 3

foreach element 4

foreach element 5

hasNext 1

hasNext 2

hasNext 3

hasNext 4

hasNext 5

- (2) LinkedList class
- Contains duplicate element
 - maintains insertion order
 - non synchronized
 - Can be used as list, stack, queue

⑤

Syntax

`LinkedList <Datatype> Variable-name = new LinkedList <datatype>();`

Important methods

- add(element)
- add(index, element)
- addAll()
- addFirst()
- addLast()
- clear()
- contains()
- get()
- getFirst()
- getLast()
- indexOf()
- lastIndexOf()
- remove()
- size()

Example

```
import java.util.*;  
public class LinkedList1 {  
    public static void main (String args[]) {  
        LinkedList <String> al = new LinkedList <String>();  
        al.add("apple");  
        al.add("banana");  
        al.add("mango");  
        al.add("allu");  
        Iterator <String> itr = al.iterator();  
        while (itr.hasNext()) {  
            Syso(itr.next());  
        }  
    }  
}
```


- (3) Vector → dynamic array to store the data elements
 → Similar to ArrayList
 → Synchronized

(6)

Syntax :-

Vector < data type > = Variable name = new Vector < variable name > ();

Example :-

```
vector <String> v = new Vector <String> ();
v.add ("dolly");
v.add ("Pinky");
v.add ("Doggy");
Iterator <String> itr = v.iterator();
while (itr.hasNext()) {
    Syso (itr.next());
}
```

Output

dolly
 Pinky
 Doggy

- (4) Stack → Subclass of Vector
 → Last in First out
 → Contains all methods of Vector & also peek(), push(), pop()

Syntax :-

Stack < data type > variable-name = new Stack < data type > ();

Example :-

```
Stack <String> Stack = new Stack <String> ();
Stack.push ("dolly");
Stack.push ("Pinky");
Stack.push ("doggy");
Stack.pop();
Iterator <String> itr = Stack.iterator();
while (itr.hasNext()) {
    Syso(itr.next());
}
```

Output

dolly
 Pinky

Queue Interface → FIFO
 → class ~~Priority Queue~~
 → Array deque

(1) Priority Queue → class of collection framework
 → FIFO
 → processed according to the priority

Syntax

PriorityQueue<String> queue = new PriorityQueue<String>();

PriorityQueue<datatype> variable name = new PriorityQueue<>();

Important method

add()

• offer() → add element

• remove()

• poll() → remove element

• element()

• peek() → next element in the Queue

Example

Queue<Integer> pq = new PriorityQueue<>();

pq.offer(40);

pq.offer(12);

pq.offer(24);

pq.offer(36);

sysout(pq);

pq.poll();

sysout(pq);

sysout(pq.peek());

add comparator
 (to change priority)
 Comparator.reverseOrder() - example

min heap
 priority
 Queue

{ [12, 36, 24, 40]
 [24, 36, 40]

24

(2) Array deque (de-e-ik)

8

→ double ended Queue

→ elements can be added/removed from front & back
both side

→ used in sliding window

Example

```
ArrayDeque<Integer> adq = new ArrayDeque<>(1);
```

```
adq.offer(23); //add
```

```
adq.offerFirst(12); // add element at first position
```

```
adq.offerLast(45); // add element at Last position
```

```
System.out.println(adq);
```

```
adq.offer(26);
```

```
System.out.println(adq);
```

```
System.out.println(adq.peek());
```

```
System.out.println(adq.peekFirst());
```

```
System.out.println(adq.peekLast());
```

```
System.out.println(adq.poll());
```

```
System.out.println("Poll()" + adq);
```

```
System.out.println(adq.pollFirst());
```

```
System.out.println("poll()" + adq);
```

```
System.out.println(adq.pollLast());
```

```
System.out.println("poll()" + adq);
```

O/P

[12 23 45]

[12 23 45 26]

12

12

26

12

poll() [23 45 26]

23

pollFirst() [45 26]

26

pollLast() [45]

Set Interface

- extends from the Java collections framework. (a)
- unordered set of elements. (a)
- duplicate not allowed

HashSet

LinkedHashSet

TreeSet

(1) HashSet

No duplicate elements, Random order $\rightarrow O(1)$

Syntax

Set <data type> Variable-name = new HashSet<>();

Example

Set <Integer> set = new HashSet<>();

set.add(32);

set.add(2);

set.add(50);

set.add(2);

set.add(65);

System.out.println(set);

set.remove(50);

System.out.println(set);

System.out.println(set.contains(100));

System.out.println(set.isEmpty());

System.out.println(set.size());

System.out.println(set.clear());

(32, 65, 2, 2, 50)

(32, 65, 2, 2)

False

False

4

[]

(2) Linked Hashset

- No duplicate element
- Same order

Set<Integer> set = new LinkedHashSet<>();

```
set.add(32);
set.add(2);
set.add(54);
set.add(21);
set.add(65);
```

System.out.println(set);

set.remove(54);

System.out.println(set);

System.out.println(set.contains(21));

System.out.println(set.isEmpty());

System.out.println(set.size());

System.out.println(set.clear());

O/P

[32, 2, 54, 21, 65]

[32, 2, 21, 65]

true

false

4

[]

(3) Treeset → Sorted order → No duplicate element

→ $O(\log n)$

Set<Integer> set = new TreeSet<>();

```
set.add(32);
set.add(2);
set.add(54);
set.add(21);
set.add(65);
System.out.println(set);
set.remove(54);
System.out.println(set);
```

System.out.println(set.contains(21));

System.out.println(set.isEmpty());

System.out.println(set.size());

System.out.println(set.clear());

O/P

[2, 21, 32, 54, 65]

[2, 21, 32, 65]

true

false

4

[]

Map

→ Stores key-value pair
→ Key unique | value may not be
overide

11

- HashMap
- LinkedHashMap
- Hashtable
- TreeMap

(1) HashMap

Syntax

```
Map <key-datatype, value-datatype> numbers = new HashMap <>();
```

example

```
Map <String, Integer> numbers = new HashMap <>();
```

```
numbers.put("one", 1);
```

```
numbers.put("two", 2);
```

```
numbers.put("three", 3);
```

```
sysout(numbers);
```

```
numbers.put("two", 23);
```

```
sysout(numbers);
```

```
numbers.putIfAbsent("two", 23);
```

```
sysout(numbers);
```

```
for (Map.Entry <String, Integer> e: numbers.entrySet()) {
```

```
    sysout(e);
```

```
    // sysout(e.getKey());
```

```
    // sysout(e.getValue()); }
```

```
sysout(numbers.containsKey(3));
```

```
sysout(numbers.containsKey("two"));
```

```
sysout(numbers.isEmpty());
```


(2) TreeMap → Sorted Order based on key
ex:- string - sorted in Alphabetical order

12

```
Map<String, Integer> numbers = new TreeMap<>();
```

```
numbers.put("one", 1);
```

```
numbers.put("two", 2);
```

```
numbers.put("three", 3);
```

```
numbers.put("four", 4);
```

```
numbers.put("five", 5);
```

```
System.out.println(numbers);
```

```
System.out.println(numbers.containsKey(3));
```

```
System.out.println(numbers.isEmpty());
```

```
System.out.println(numbers.remove("three"));
```

```
System.out.println(numbers);
```

O/P

{ five = 5, four = 4, one = 1,
three = 3, two = 2 }

four

five

three

{ five = 5, four = 4, one = 1,
two = 2 }

(3) LinkedHashMap

(4) Hashtable

Array class

This Array class is used to manipulate over arrays (primitive)

1) Search in an array

```
int[] numbers = { 1, 2, 3, 4, 5, 6, 7, 8, 10 };
```

```
int index = Arrays.binary search(numbers, key: 4);
```

```
System.out.println("the index of element 4 in the array is " + index);
```

O/P :- the index of element 4 in the array is 3

2) sort an array

```
Integer[] numbers = { 10, 2, 32, 12, 15, 76, 17, 48, 79, 9 };
```

```
Arrays.sort(numbers);
```

```
for (int i : numbers) {
```

```
    System.out.print(i + " ");
```

```
}
```

O/P :- 2 9 10 12 15 17 32 48 76 79

3)

```
Integer[] num = { 10, 2, 32, 12, 15, 76, 17 };
```

```
Arrays.fill(numbers, 12);
```

```
for (int i : numbers) {
```

```
    System.out.print(i + " ");
```

```
}
```

O/P :- 12 12 12 12 12 12 12

Collections class

14

```
list <Integer> list = new ArrayList<>();
```

```
list.add(34);
```

```
list.add(12);
```

```
list.add(9);
```

```
list.add(76);
```

```
list.add(29);
```

```
list.add(75);
```

```
list.add(9);
```

```
Sysout("min element " + collections.min(list));
```

```
Sysout("max element " + collections.max(list));
```

```
Sysout("Frequency " + collections.frequency(list, 9));
```

```
collections.sort(list);
```

```
Sysout(list);
```

```
collections.sort(list, Comparator.reverseOrder());
```

```
Sysout(list);
```

O/P

min element 9

max element 76

frequency 2

9 9 12 29 34 75 76

76 75 34 29 12 9 9